

AI Machine Learning Engineer

SUMMARY

The objective of this project is to build an AI Machine Learning Engineer capable of performing the work of an experienced ML engineer. The proposed system aims to automate the entire ML engineering workflow, not just model training, for classical machine learning (tabular data).

PROJECTS

AI Machine Learning Engineer | An agentic AutoML system for end-to-end machine learning development.

- Builds an AI Machine Learning Engineer capable of performing the work of an experienced ML engineer.
- Automates requirement understanding, dataset analysis, planning, pipeline building, hyperparameter optimization, training, evaluation, explainability, and report generation.
- Supports classical machine learning on tabular data with CSV, Excel, and Parquet formats.
- Handles Binary Classification, Multi-class Classification, and Regression tasks.
- Implements a multi-agent architecture: Requirement Understanding Agent, Dataset Analysis Agent, Planning Agent, Pipeline Builder, Hyperparameter Agent, Training Agent, Evaluation Agent, and Report Generator.
- Uses Scikit-learn Pipelines for preprocessing, feature engineering, and model training.
- Uses Optuna for hyperparameter search and optimization.
- Trains and compares Random Forest, XGBoost, CatBoost, LightGBM, and Logistic Regression.
- Generates evaluation outputs including Accuracy, Precision, Recall, F1 Score, ROC-AUC, Confusion Matrix, Feature Importance, and SHAP explanations.
- Produces a complete PDF report with training summary, best hyperparameters, evaluation graphs, recommendations, and deployment code.
- Technology stack includes FastAPI backend, React frontend, Pandas, Scikit-learn, Optuna, SHAP, MLflow, PostgreSQL, pgvector, and LangGraph.
- Recommends architecture separating deterministic Python computation from LLM reasoning using local LLMs (Ollama + Qwen/Llama).

TECHNICAL SKILLS

FastAPI

React

Pandas

Scikit-learn

Optuna

SHAP

MLflow

PostgreSQL

pgvector

LangGraph

Random Forest

XGBoost

CatBoost

LightGBM

Logistic Regression

Binary Classification

Multi-class Classification

Regression

Scikit-learn Pipelines

Hyperparameter Optimization
Experiment Tracking
Explainability
Ollama
Qwen
Llama
Gemma
DeepSeek
Mistral
vLLM
llama.cpp
GGUF
Quantization
SIMD

SCOPE

Intentionally limited to Classical Machine Learning (Tabular Data). Supported dataset formats: CSV, Excel, Parquet. Supported ML tasks: Binary Classification, Multi-class Classification, Regression. Future extensions include Time Series, Clustering, Anomaly Detection, Image Models, and NLP Models.

AGENT ARCHITECTURE DETAILS

Specialized agents include Requirement Agent, Dataset Agent, Planning Agent, Pipeline Builder, Hyperparameter Agent, Training Agent, Evaluation Agent, and Report Agent, each with defined responsibilities for autonomous ML workflow execution.

LLM DEPLOYMENT OPTIONS

Three options described: API-based LLM (GPT/Claude/Gemini), Local LLM (Qwen/Llama/Gemma/DeepSeek/Mistral via Ollama), and Local LLM with vLLM for high-throughput multi-user serving.

INFRASTRUCTURE COMPARISON

Includes comparison of Ollama vs vLLM vs llama.cpp across beginner friendliness, offline capability, API server support, multi-user performance, CPU/GPU support, and best use cases.

FUTURE EXTENSIONS

Support image datasets, NLP datasets, time-series forecasting, multi-modal pipelines, NAS, automatic deployment, self-improving agent memory, reinforcement learning for pipeline optimization, multi-agent debate, cloud deployment with Kubernetes, and enterprise MLOps integration.

FINAL GOAL

Create an Agentic AI Machine Learning Engineer that reasons, makes decisions, and automates the complete machine learning development lifecycle for classical ML tasks, from problem understanding to deployment recommendation.